Authentication & Authorization Design

Authentication Requirements

Users must be able to:

- Register an account
- Login securely
- Remain authenticated across sessions
- Logout safely
- Refresh access tokens without re-login

Authentication Flow

User → Register / Login Endpoint → JWT Access + Refresh Tokens Issued → Tokens Stored in HTTP-only Cookie → Protected Routes via Middleware → Token Verification

---

Token Strategy

Access Token

- Short-lived JWT
- Used for API authentication
- Verified using middleware

Refresh Token

- Long-lived JWT
- Stored hashed in database
- Used to generate new access tokens

Why JWT?

Chosen Approach: JWT Authentication

Alternatives:

- Session-based authentication
- Opaque tokens

Reasons:

- Stateless authentication (scales well)
- No server-side session storage required
- Works well with distributed systems

Tradeoffs:

- Token revocation is more complex
- Requires refresh token strategy

Why Cookies?

HTTP-only Cookie Storage

Tokens are stored in:

- HTTP-only cookies
- Secure flag in production

Reasons:

- Protects against XSS attacks
- Prevents token access via JavaScript
- Simplifies client-side handling

Alternatives:

- LocalStorage (rejected due to XSS risk)

Authorization Strategy

Role-Based Access Control (RBAC)

Roles:

- Store Owner → manage stores and products
- User → read-only access

Middleware Enforcement

- JWT middleware validates token
- Attaches user payload to request
- Protects private routes

Security Measures

- Password hashing using bcrypt
- Rate limiting on auth endpoints (token bucket via Redis)
- JWT expiration strategy
- Input validation middleware
- Secure cookie configuration
- Hashed refresh tokens stored in database

Future Improvements

- Refresh token rotation on every use
- OAuth2 / Social login integration
- Multi-factor authentication (MFA)
- Device/session management system